SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA0411
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I K. A. Analiya (192521395) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K. A. Analiya

ANNEXURE A

SHORT ANSWER TEST

Question 1 (5 Marks)

Suppose a modern Operating System supports multiple users and processes with shared resources like CPU, memory, and I/O devices.

(a) Classify different types of Operating Systems suitable for this scenario and justify your choice.

The following types of Operating Systems are suitable:
1. Multi-user OS: Allows multiple users to access the system simultaneously.
2. Multitasking OS: Executes multiple processes by sharing CPU time.
3. Time-sharing OS: Allocates CPU time to each process for quick response.
4. Multiprocessing OS: Uses multiple processors to execute tasks efficiently.

Justification: A Multi-user Time-sharing OS is most suitable because it allows multiple users to share CPU, memory, and I/O devices efficiently while maintaining resource protection.

(b) Draw the structure of an Operating System showing how resources are managed.

Users / Applications
↓
System Call Interface
↓
Operating System Kernel
↓
Process Management
Memory Management
File Management
Device Management
↓
Hardware Resources (CPU, Memory, I/O)

Explanation: The OS kernel manages CPU, memory, files, and I/O devices. It acts as an interface between applications and hardware to ensure efficient resource management.

(c) Write a sequence of system calls required to create and execute a new process in such an environment. **(BL2)**

The sequence of system calls required to create and execute a new process is:
1. fork() – Creates a new child process.
2. exec() – Executes a new program in the child process.
3. wait() – Makes the parent process wait for child completion.
4. exit() – Terminates the child process.

System Call Sequence:
Start → fork() → Child Process → exec() → Execute Program → exit() → Parent wait() → End.

Explanation: The fork() call creates a child process, exec() loads a new program, exit() terminates the child, and wait() allows the parent to collect its completion status.

Question 2 (5 Marks)

Consider an Operating System managing multiple files and devices for concurrent users.

(a) Categorize system calls used for file and device management.

1. File Management System Calls:
- create() – Creates a new file.
- open() – Opens an existing file.
- read() – Reads data from a file.
- write() – Writes data into a file.
- close() – Closes an opened file.
- unlink() – Deletes a file.

2. Device Management System Calls:
- request() – Requests access to a device.
- release() – Releases an allocated device.
- read() – Reads data from a device.
- write() – Sends data to a device.
- ioctl() – Controls device-specific operations.

These system calls help the OS manage files and devices efficiently while supporting multiple users.

(b) Illustrate how the OS handles file operations using a layered structure diagram.

Layered Structure of File Operations:

User / Application Programs ↓ System Call Interface (open, read, write, close) ↓ Logical File System ↓ File Organization Module ↓ Basic File System ↓ I/O Control (Device Drivers) ↓ Storage Hardware (HDD / SSD)

Explanation: The application requests file operations through system calls. The OS processes these requests through different layers and communicates with storage devices to perform file operations efficiently and securely.

(c) Write a pseudo-code or C-based system call sequence to open, read, and close a file. **(BL3)**

```
#include <fcntl.h>
#include <unistd.h>
int main() {
   char buffer[100];
   int fd = open("file.txt", O_RDONLY);
   if (fd < 0)
      return 1;
   read(fd, buffer, sizeof(buffer));
   close(fd);
   return 0;
}
```

Sequence: Start → open() → read() → close() → End.

Question 3 (5 Marks)

Assume a real-time Operating System used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time applications.

Real-Time Operating System (RTOS)

The suitable OS is a Hard Real-Time Operating System (Hard RTOS), which ensures tasks are completed within strict deadlines.

Reasons for Suitability:
1. Fast Response: Responds quickly to external events.
2. Task Scheduling: Executes high-priority tasks on time.
3. Reliability: Provides stable and predictable performance.
4. Deadline Management: Ensures critical tasks finish within specified time limits.
5. Resource Management: Efficiently manages CPU, memory, and I/O devices.

Hard RTOS is suitable for industrial machinery because it provides timely and reliable control of critical operations.

(b) Draw a diagram representing the interaction between hardware, kernel, and user processes.

Interaction Between Hardware, Kernel, and User Processes:

User Processes (Task 1, Task 2, Task 3) ↓ System Call Interface ↓ Real-Time Operating System Kernel ↓
Task Scheduling	Process Synchronization
↓ Device Drivers ↓ Hardware (CPU, Sensors, Actuators)

Explanation: The RTOS kernel acts as an interface between user processes and hardware. It manages task scheduling, synchronization, and interrupts to ensure timely execution of real-time operations.

(c) List and explain system calls required for process synchronization in this system. **(BL2)**

System Calls for Process Synchronization:

1. sem_init() – Initializes a semaphore for process synchronization.
2. sem_wait() – Makes a task wait until a shared resource becomes available.
3. sem_post() – Releases the semaphore and allows another waiting task to proceed.
4. pthread_mutex_lock() – Locks a shared resource to prevent simultaneous access.
5. pthread_mutex_unlock() – Unlocks the resource after completing the task.

Explanation: These synchronization operations prevent race conditions and ensure safe resource sharing between real-time tasks in industrial machinery.

Question 4 (5 Marks)

A distributed computing system consists of multiple interconnected computers sharing resources over a network.

(a) Classify the type of Operating System used and compare it with centralized OS.

Distributed Operating System (Distributed OS)

A Distributed OS manages multiple interconnected computers and allows them to share resources over a network as a single system.

Comparison between Distributed OS and Centralized OS:

Distributed OS	Centralized OS
Uses multiple connected computers.	Uses a single central computer.
Shares resources across different nodes.	Resources are managed centrally.
Distributes workload among systems.	Workload is handled by one system.
Failure of one node may not affect the entire system.	Central system failure can stop all operations.
Provides better scalability.	Has limited scalability.

Conclusion: A Distributed OS is suitable because it enables efficient resource sharing, workload distribution, and communication between interconnected computers.

(b) Draw the architecture of a distributed OS showing communication between nodes.

Node 1 (Computer 1) ↔ Node 2 (Computer 2) ↕ ↕ Communication Network ↕ ↕ Node 3 (Computer 3) ↔ Node 4 (Computer 4)

Each Node Contains:

Processor (CPU) → Local Memory → Operating System → Network Interface

Explanation: All nodes are connected through a communication network. The Distributed OS coordinates communication, resource sharing, and task execution between nodes, making them function as a unified system.

(c) Write the sequence of system calls involved in inter-process communication (IPC). **(BL3)**

- socket() – Creates a communication endpoint.
- bind() – Assigns an address to the socket.
- listen() – Waits for incoming connection requests.
- accept() – Accepts a connection from another process.
- connect() – Establishes a connection with another node.

Question 5 (5 Marks)

Consider a scenario where an Operating System must manage process scheduling, memory allocation, and device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

Major Functions of an Operating System:
1. Process Management: Schedules processes and allocates CPU time for efficient execution.
2. Memory Management: Allocates and deallocates memory for running programs.
3. Device Management: Controls input/output devices using device drivers.
4. Resource Allocation: Distributes CPU, memory, and devices among processes.
5. Communication Management: Enables data exchange between processes and connected devices.
These functions ensure efficient resource utilization and smooth system performance.

(b) Draw a block diagram representing OS components and their interactions.

Users / Application Programs
↓
System Call Interface
↓
Operating System Kernel
↓
Process Scheduler
Memory Manager
Device Manager
File Manager
↓
Device Drivers
↓
Hardware (CPU, Memory, I/O Devices)
Explanation: The OS kernel coordinates process scheduling, memory allocation, and device communication. It connects application programs with hardware through system calls and device drivers.

(c) Write system call examples for memory allocation and device management. **(BL3)**

1. Memory Allocation System Calls:

- brk() – Changes the size of the process data segment.

- mmap() – Allocates memory by mapping memory pages.

- munmap() – Releases previously mapped memory.

2. Device Management System Calls:

- open() – Opens a device for access.

- read() – Reads data from a device.

- write() – Sends data to a device.

- ioctl() – Performs device-specific control operations.

- close() – Closes the device.

Example Sequence:

Memory: mmap() → Use Memory → munmap()

Device: open() → read() / write() → ioctl() → close()

RUBRICS FOR CALCULATION

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand